

Fast and Parallel Decomposition of Constraint Satisfaction Problems

IJCAI-PRICAI 2020

Georg Gottlob^{1,2} Cem Okulmus² Reinhard Pichler²

¹University of Oxford ²TU Wien

Motivation

Three Problems

- Conjunctive Query Evaluation
- **Constraint Satisfaction Problem**
- Homomorphism Problem

All these problems are essentially the same

All these problems are based on **hypergraphs**.

Problem

Is there any teacher having a child enrolled in her course?

$$ans \leftarrow Enrolled(S, R, C) \wedge Teaches(P, A, C) \wedge Parent(P, S)$$

Constraint Scope

Enrolled	Teaches	Parent
$\{(S, R, C)\}$	$\{(P, A, C)\}$	$\{(P, S)\}$

Constraint Relation

Enrolled	Teaches	Parent
$\{(Stuart, true, Logic),$ $(Susan, false, Physics)\}$	$\{(Martha, true, Physics),$ $(Thomas, false, Logic)\}$	$\{(Martha, Stuart),$ $(Thomas, Susan)\}$

Problem

Is there any teacher having a child enrolled in her course?

$$ans \leftarrow Enrolled(S, R, C) \wedge Teaches(P, A, C) \wedge Parent(P, S)$$

Constraint Scope

Enrolled	Teaches	Parent
$\{(S, R, C)\}$	$\{(P, A, C)\}$	$\{(P, S)\}$

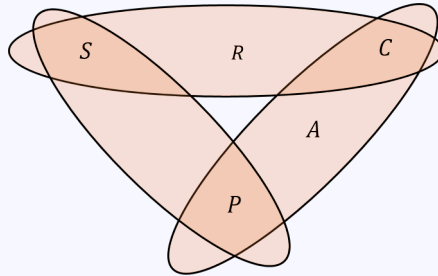
Constraint Relation

Enrolled	Teaches	Parent
$\{(Stuart, true, Logic),$ $(Susan, false, Physics)\}$	$\{(Martha, true, Physics),$ $(Thomas, false, Logic)\}$	$\{(Martha, Stuart),$ $(Thomas, Susan)\}$

CSP and Hypergraphs (cont.)

Problem

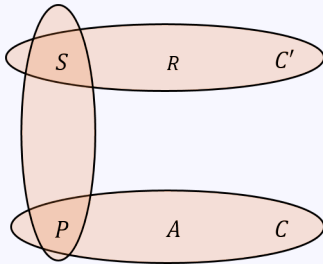
Is there any teacher having a child enrolled in her course?

$$ans \leftarrow Enrolled(S, R, C) \wedge Teaches(P, A, C) \wedge Parent(P, S)$$


Problem

Is there any teacher having a child enrolled in **some** course ?

$$ans \leftarrow Enrolled(S, R, C') \wedge Teaches(P, A, C) \wedge Parent(P, S)$$



Acyclic CSPs (cont.)

Problem

Is there any teacher having a child enrolled in **some** course ?

$$ans \leftarrow Enrolled(S, R, C') \wedge Teaches(P, A, C) \wedge Parent(P, S)$$

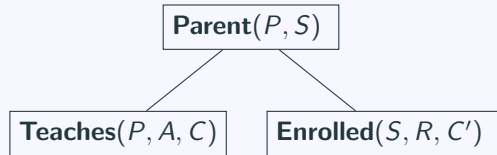
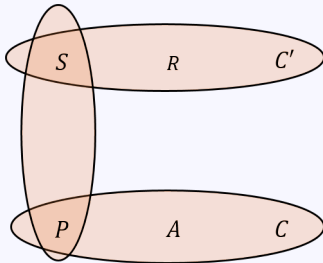


Figure 1: Join Tree

- **NP-complete in the general case**
(Schaefer '78)
- **Tractable in case of acyclic CSPs**
(Yannakakis '81, Gyssens et al. '94)

Interest in larger tractable classes of CSPs

Complexity of CSP Evaluation (cont.)

Problem

Is there any teacher having a child enrolled in her course?

$$ans \leftarrow Enrolled(S, R, C) \wedge Teaches(P, A, C) \wedge Parent(P, S)$$

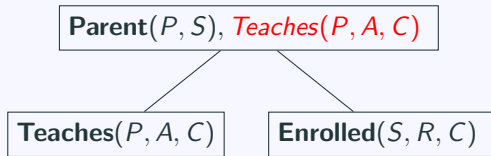
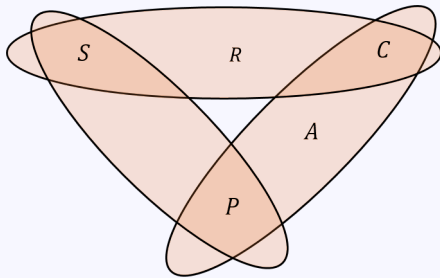
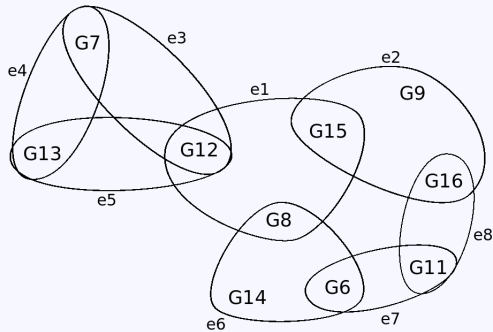


Figure 2: Hypergraph Decomposition

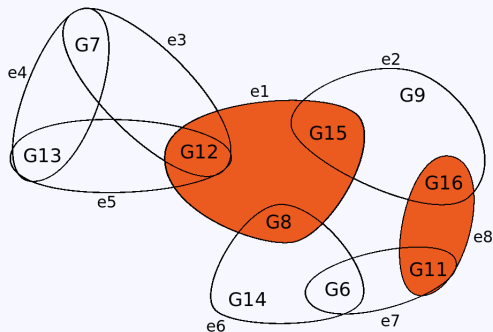
Decomposition Algorithm

Decomposition Algorithm (1/5)



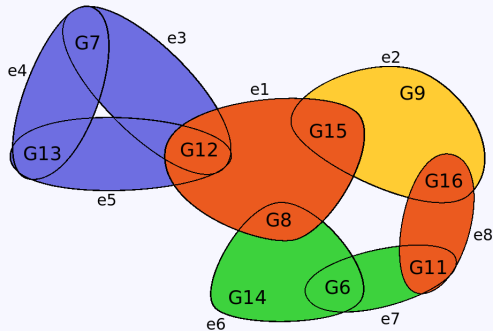
- Finding a cut (separation)

e1, e8



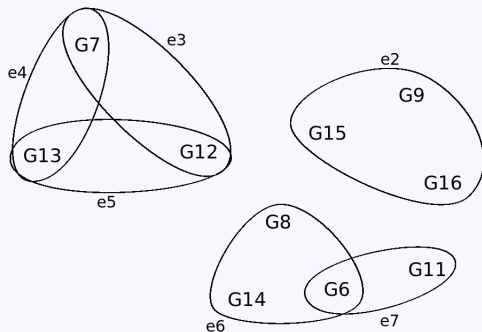
- Finding a cut (separation)

e1, e8



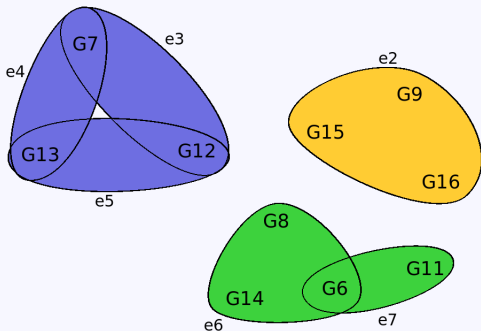
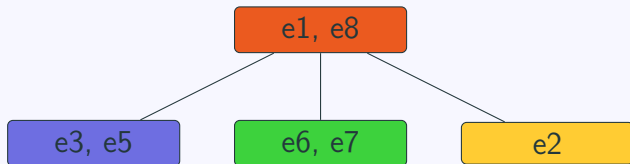
- new connected components

e1, e8



- continue recursively on components

Decomposition Algorithm (5/5)



- continue recursively on components

Balanced Separator

A separator S s.t. all for all its connected components C_i $1 \leq i \leq n$ we have $|C_i| \leq \frac{|E(H)|}{2}$ is called a **balanced separator**

Theorem (Adler et al. '07)

A hypergraph H has **no** balanced separator of size $\leq k$
 $\Rightarrow H$ has no GHD of width $\leq k$

- Used by Fischl et al. (PODS 2019) to compute Generalized Hypertree Decompositions

- Practical tools for hypergraph decomposition ... (Gottlob, Samer '09, Fischl et al. '19)
 - Can be found at github.com/TUfischl/newdetkdecomp
- ... and large benchmark to test them with. (Fischl et al. '19)
 - **HyperBench**, accessible at hyperbench.dbai.tuwien.ac.at
- However, **large number of CSPs not decomposable**.
 - Over 50 % of CSPs not minimally decomposable (due to time-outs)

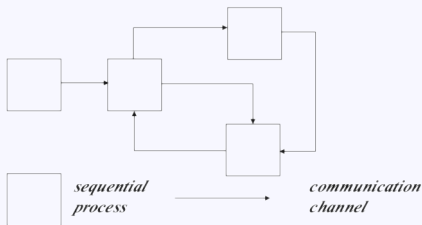
- Provide a number of key algorithmic improvements
 - Hypergraph Preprocessing
 - Improved Search for Balanced Separators
- Using parallelisation to speed up computation of hypergraph decompositions
 - Using Go, and its support for concurrency
 - New parallel algorithm for computing hypergraph decompositions

The decomposer can be found in `github.com/cem-okulmus/BalancedGo`

Parallelisation

Parallelisation

- Implemented in Go (designed by Google in 2009)
 - concurrent programming scheme (goroutines)
- Challenge: finding a **good design** for parallelising the algorithm
 - The parallel search for balanced separators
 - The recursive calls on resulting components



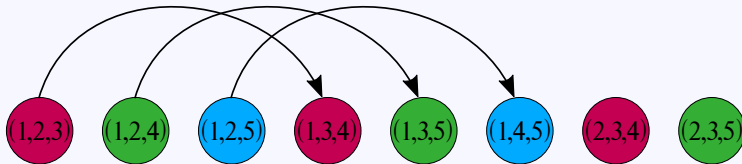
Our parallel implementation

Recursive Calls

Made easy by use of goroutines.

Search for Separators

- Worker and Central node model
- Each Worker has its own search space (not waiting on other workers)
- If Worker finds a balanced separator, entire search is paused
 - central node keeps track of the search progress, for fast backtracking



Experimental evaluation

Setup

- Various tests on instances from HyperBench¹
- 20 node cluster, 24-core Intel Xeon E5-2650v4 CPU@2.20GHz & 256 GB RAM
- Each test with 1 hour timeout

Results

- Increase of over 70 % of optimally decomposed CSPs: from 825 to **1430**, for 2035 total
- For CSPs from practice, increase of over 100 %: from 386 to **763**, for 1090 total
- In total, around 70 % of all CSPs in HyperBench now optimally decomposed: **1430** out of 2035

¹hyperbench.dbai.tuwien.ac.at

Experimental evaluation (cont.)

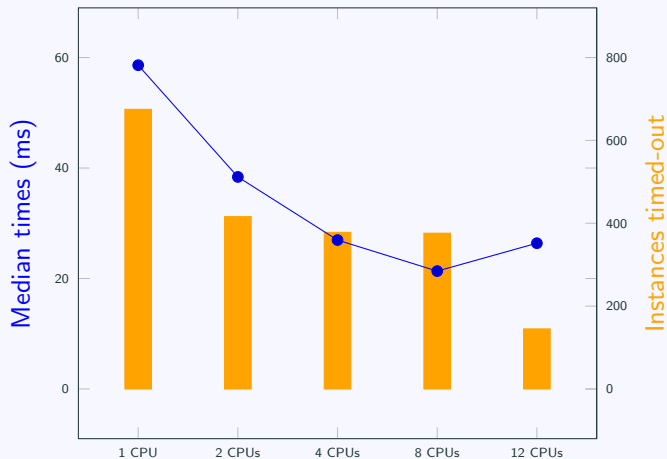


Figure 3: Study of the speedup w.r.t. number of CPUs used

What this means & Follow Ups

- Laying foundations for more systems and applications to evaluate CSPs based on their **structural properties**.

Future Work

- Parallel Algorithms for other types of decomposition
 - HDs instead of GHDs
 - computable in polynomial time
- Looking into Cloud Computing, both for decomposition, and evaluation of CSPs
 - Massively concurrent algorithms possible

Thank you for your attention